G1 Test Protocol — Operator Runbook

Rev. 2026-08-07 · Every command is single-line and paste-ready. Two lanes: TWIN (simulator, any day) and REAL (lab session, R1 first). Source of this document: `docs/src/` — regenerate with `tools/build_docs.sh`.

The three iron rules. One change per batch · No code edits during sessions or while a campaign runs · Instrumentation down = run invalid, repeat it.

PART 0 — Session setup (both lanes)

0.1 Git sync (do this before anything, in this exact order)

```
on GPUEDGE: cd <repo> && git add goto.log && git commit -m "goto.log GPUEDGE appends" && git pus
h
```

```
cd "/Users/adrianlendinezibanez/Claude/Projects/G1 ROBOT" && git pull --rebase && git push origi
n tutor-feedback-metareasoner
```

```
back on GPUEDGE: git pull --rebase
```

If a conflict appears on `results/runs_stats.csv` or `goto.log`: resolve by keeping BOTH sides (union), never discard lines.

0.2 Choose your code

Purpose	Checkout
Twin tests of the meta machine / noise / camera	`git checkout tutor-feedback-metareasoner-sim`
REAL R1 re-baseline (Gate 1)	`git checkout golden-doorvis` (detached HEAD is fine)
Everything else real (post-Gate-1, per plan)	`git checkout main`

Repository layout note. Analysis tools live in `analysis/`, campaign runners in `campaigns/`, reasoner configs in `config/`, trust state in `state/`, maps and waypoints in `data/`. Config and state are still passed as **bare filenames** — the loader resolves them, so the commands below work unchanged.

PART 1 — TWIN lane (simulator)

1.1 Infrastructure up

```
docker ps | grep g1sim
```

If the container was restarted, relaunch Gazebo AND rosbridge (both are manual):

```
docker exec -d g1sim bash -c "source /opt/ros/humble/setup.bash && source /home/ubuntu/g1_ws/ins
tall/setup.bash && ros2 launch g1_sim lab.launch.py gui:=false > /tmp/lab_launch.log 2>&1"
```

```
docker exec -d g1sim bash -c "source /opt/ros/humble/setup.bash && source /home/ubuntu/g1_ws/ins
tall/setup.bash && ros2 launch rosbridge_server rosbridge_websocket_launch.xml port:=8765"
```

If noVNC port 6080 is stolen: `docker stop ros2_humble`. Watch the simulation at `http://localhost:6080/vnc_lite.html` (password `ubuntu`) — data runs go headless.

1.2 Teleports (ALWAYS teleport before a manual run)

```
docker exec g1sim bash -c "gz model -m g1 -x 0.99 -y 0.57 -z 0.05 -Y -2.1"      # place at A
```

```
docker exec g1sim bash -c "gz model -m g1 -x -4.75 -y 2.60 -z 0.05 -Y -0.9"     # place at B
```

1.3 The standard run command (build it from blocks)

Base env (always) — use a fresh `G1_M2_STATE` per experiment; delete the file to restart learning:

```
G1_META2=2 G1_M2_CFG=config_meta2_g1payload.json G1_SIM_ID=lab_v1_test G1_M2_STATE=meta2_state_t
est.json G1_M2_L3=1 G1_M2_L4=1 G1_M2_PROFILES=1 G1_M2_DOORLIB=1 G1_M2_FRAGSPEED=1 G1_M2_ABORT_WI
N=150 G1_M2_ABORT_PROG=0.25 G1_M2_HELP_S=16
```

Optional blocks: noise `G1_SIM_NOISE=1` · synthetic camera + DOOR-VIS `G1_SIM_PERC=1 G1_PERC=127.0.0.1:8010 G1_DOOR_VIS=1` · machine off `G1_METASM=0 G1_RETREAT=0`. Then append the runner:

```
/Users/adrianlendinezibanez/unitree_webrtc_connect/.venv/bin/python g1_sim_adapter.py goto B
```

Example — full golden config in the twin (noise + camera + vision + machine), B→A:

```
cd "/Users/adrianlendinezibanez/Claude/Projects/G1 ROBOT" && G1_SIM_NOISE=1 G1_SIM_PERC=1 G1_PER
C=127.0.0.1:8010 G1_DOOR_VIS=1 G1_META2=2 G1_M2_CFG=config_meta2_g1payload.json G1_SIM_ID=lab_v1
_test G1_M2_STATE=meta2_state_test.json G1_M2_L3=1 G1_M2_L4=1 G1_M2_PROFILES=1 G1_M2_DOORLIB=1 G
1_M2_FRAGSPEED=1 G1_M2_ABORT_WIN=150 G1_M2_ABORT_PROG=0.25 G1_M2_HELP_S=16 /Users/adrianlendinez
ibanez/unitree_webrtc_connect/.venv/bin/python g1_sim_adapter.py goto A
```

1.4 The five twin tests

#	Test	How	PASS if
T1	Clean baseline, machine ON	Teleport A → run goto B (base env only)	**REACHED 95–115 s, 0 col, 0 retreats; NORMAL 55–60% / BLIND 40–45%**
T2	Human collision reports → DEGRADED	During a T1 run, send **3× hcol ~4 s apart** (command below)	**laser_lied ×3, trust ≈0.30, one retreat "por col", DEGRADED visible, then recovery**
T3	Trap → RECOVERY → ASSIST → human	Teleport B, spawn box at door, goto A; when it prints ASISTENCIA: delete box + send hmove:40	**3 retreats → full stop → "asistencia RECIBIDA" → budget re-armed → resumes**
T4	Closed-loop A/B campaigns	`python3 -u campaigns/sim_ab_msm_campaign.py` then `python3 -u campaigns/sim_ab_msm_noise_campaign.py` (~1 h each)	**base: both arms 6/6; noise: OFF ≈3/6 + many col vs ON 6/6 + 0 col**
T5	Full golden config (noise+camera+vision)	Example command in §1.3	**REACHED ~100–110 s, "DOOR-VIS activo", door_crossed ≈35–45 s**

Injection commands (second terminal):

```
python3 -c "import socket,time; s=socket.socket(socket.AF_INET,socket.SOCK_DGRAM); [ (s.sendto
(b'hcol',('127.0.0.1',7777)), time.sleep(4)) for _ in range(3) ]"
```

```
python3 -c "import socket; socket.socket(socket.AF_INET,socket.SOCK_DGRAM).sendto(b'hmove:40',
('127.0.0.1',7777))"
```

Trap box spawn / delete:

```
docker exec g1sim bash -c "source /opt/ros/humble/setup.bash && ros2 service call /spawn_entity
gazebo_msgs/srv/SpawnEntity \"{name: 'trampa_msm', xml: '<sdf version=\\\"1.6\\\"><model name=
\\\"trampa_msm\\\"><static>true</static><link name=\\\"l\\\"><collision name=\\\"c\\\"><geometry
><box><size>0.45 0.45 0.5</size></box></geometry></collision><visual name=\\\"v\\\"><geometry><b
ox><size>0.45 0.45 0.5</size></box></geometry></visual></link></model></sdf>', initial_pose: {po
sition: {x: -3.30, y: 0.60, z: 0.25}}}\""
```

```
docker exec g1sim bash -c "source /opt/ros/humble/setup.bash && ros2 service call /delete_entity
gazebo_msgs/srv/DeleteEntity \"{name: 'trampa_msm'}\""
```

1.5 Analysis after every twin run

```
python3 analysis/analyze_msm.py "$(ls -t dataset/2026*_ours_B.json | head -1)"
```

```
python3 analysis/replay_msm.py dataset/<any_run>.json
```

Noise-lane results are their own baseline — never compare noisy times against no-noise times, and never either against real-robot times.

PART 2 — REAL lane (lab session; R1 first)

2.1 GPUEDGE: perception server

First, check the GPUs are free. A local LLM service (llama-server / Kimi) lives on the same box and can hold ~20 GB on *each* card, leaving too little for the depth model. Do this before the robot is even switched on:

```
nvidia-smi --query-gpu=memory.used,memory.total --format=csv
```

If both cards are near full, stop the LLM service (or wait for it) before launching perception — discovering this mid-session, with the cup full, is the expensive way to find out.

Then launch the server (this is the exact command in use):

```
python perception_server.py --host 0.0.0.0 --port 8008 --fx 300 --fy 300 --cx 160 --cy 90 --cam-h 1.10 --cam-pitch -10 --debug --self-mask 0.25
```

(`--debug` enables the viewer at `:8008/view`; drop it if you do not need it.) Check from the Mac:

```
curl -s http://192.168.51.80:8008/health
```

PASS: `ok: true` and MODE gpu. If detections come back empty on the smoke test, the camera frame or intrinsics are wrong — stop and fix before any run.

2.2 Robot bring-up (every boot)

1. Robot on → open the vendor app with **Bluetooth ON** → pair from the app (the AP password rotates every boot; iPhone Settings will say "incorrect password" — expected, don't fight it).
2. iPhone via USB to the Mac → `ios_webkit_debug_proxy` running → app teleop moves the robot (`g1_teleop.py`).
3. Localisation check:

   ```
   python g1_goto.py reloccheck
   ```

4. Marker up in its own terminal BEFORE the first run (v2 keys on golden: ENTER=spill, w <g>=weigh, i=invalid; branch v3 adds c and m <cm>):

   ```
   python3 spill_mark.py
   ```

   Verify its heartbeat appears in the run console.

2.3 The R1 run loop (10 runs: 5× A→B, 5× B→A, alternating)

1. Fill the cup to **247 g**. Weigh, type `w 247` in the marker.
2. Launch the run (golden real config):

```
cd "/Users/adrianlendinezibanez/Claude/Projects/G1 ROBOT" && G1_META2=2 G1_M2_CFG=config_meta2_g1payload.json G1_M2_STATE=meta2_state_r1.json G1_M2_STATE_INIT=meta2_state_twin_explored.json G1_M2_REALCAL=1 G1_M2_L3=1 G1_M2_L4=1 G1_M2_PROFILES=1 G1_M2_DOORLIB=1 G1_M2_FRAGSPEED=1 G1_M2_ABORT_WIN=150 G1_M2_ABORT_PROG=0.25 G1_M2_HELP_S=16 G1_DOOR_VIS=1 G1_PERC=192.168.51.80:8008 /Users/adrianlendinezibanez/unitree_webrtc_connect/.venv/bin/python g1_goto.py goto B
```

3. During the run: ENTER on every slosh (a burst = ONE mark). Do not touch the robot.
4. Run ends → weigh the cup → `w <grams>`. Never refill mid round-trip.
5. Instrument trouble → `i reason` and repeat the run.

6. Reverse leg: same command with `goto A`.

Stop-rule: 3 consecutive failures of the same mode → STOP the session. Data home, autopsy offline. No lab improvisation, no code edits.

2.4 Gate 1 (decide at the lab, before packing)

Check	PASS
A→B reached	≥ 90%
B→A reached	≥ 70%
Collisions	median 0 per run
Times	golden band ~50–110 s

PASS → continue with the Results Acquisition Plan. FAIL → autopsy first:

```
python3 analysis/analyze_msm.py dataset/<failed_run>.json
```

```
python3 analysis/replay_msm.py dataset/<failed_run>.json
```

2.5 End of session

```
cd "/Users/adrianlendinezibanez/Claude/Projects/G1 ROBOT" && git add -A && git commit -m "R1 ses
sion data" && git pull --rebase && git push
```

(On conflicts in append-only files: union of both sides.) Then GPUEDGE: `git pull --rebase`.

PART 3 — Quick-reference card

Action	Command
Twin alive?	`docker ps | grep g1sim`
Teleport A / B	`gz model -m g1 -x 0.99 -y 0.57 -z 0.05 -Y -2.1` · `-x -4.75 -y 2.60 -z 0.05 -Y -0.9` (inside `docker exec g1sim bash -c "…"`)
Twin run	base env + `…/.venv/bin/python g1_sim_adapter.py goto A|B`
Real run	real env (§2.3) + `…/.venv/bin/python g1_goto.py goto A|B`
Analyse newest run	`python3 analysis/analyze_msm.py "$(ls -t dataset/2026*_ours_*.json | head -1)"`
Report collision (human)	marker key `c` (branch) or the hcol one-liner (§1.4)
Give assistance	marker key `m 40` (branch) or the hmove one-liner (§1.4)
Meta-reasoner intact?	`cd meta-reasoner-2.0 && python3 -m unittest discover -q` → OK
Perception health	`curl -s http://192.168.51.80:8008/health`
Kill a stray run	`pgrep -f g1_sim_adapter` → `kill <PID>` (NEVER pkill -f)

Flag	Meaning (default)
`G1_METASM` / `G1_RETREAT` / `G1_EXIT_RETREAT`	meta state machine / retreat v2 / retreat-at-exit (all 1 on the branch)
`G1_SIM_NOISE`	calibrated sensor noise in the twin (0)
`G1_SIM_PERC` + `G1_PERC=127.0.0.1:8010` + `G1_DOOR_VIS`	synthetic camera → DOOR-VIS in the twin (0)

`G1_M2_REALCAL`	platform-calibrated QoE — REAL ROBOT ONLY (0)
`G1_M2_STATE` / `G1_M2_STATE_INIT`	per-experiment trust state / sim-to-real seed (resolved from `state/`)

Expected numbers — twin clean ON: 95–115 s · twin noise A/B: OFF ≈3/6 + ~27 col vs ON 6/6 + 0 col · golden real: B→A 52–60 s, 0 col · real spills: ~12/run at 247 g is NORMAL (REALCAL handles it).